

List of Figures

1	Code sample showing how N is randomly generated between 50 and 100.	1
2	This code part shows how cost values are set randomly from \$20 up to \$200.	1
3	Backend code that measures the speed of each algorithm.	2
4	Our implementation of the Greedy logic.	2
5	The implementation of the Hungarian algorithm.	3
6	Code that handles saving the player name to the DB.	3
7	The backend function used to persist the round results.	4
8	The SQLite database schema layout.	4
9	This shows where the user types their name before starting.	5
10	Testing our Greedy logic with pytest.	6
11	Testing the Hungarian algorithm for correctness.	7

List of Tables

1	Group Contribution and Marks	iv
---	--	----

Contents

Contribution Table	iv
1 Minimum Cost Problem	1
1.1 Functionality	1
1.1.1 Code Segment: N changes randomly from 50 to 100 in each game round	1
1.1.2 Code Segment: Cost of assigning each task to each employee changing randomly from \$20 to \$200	1
1.1.3 Code Segment: Record in the database how long it takes to find the minimum cost using each algorithm	1
1.1.4 Code Segment: Two different appropriate algorithms	2
1.1.5 Code Segment: Save player's name and their response in the database	3
1.1.6 Database Table structure for the Game	4
1.2 User Interface	5
1.2.1 UI screenshot for inputs and answers	5
1.2.2 Validations and error handling	5
1.3 System Testing	6
1.3.1 Unit Testing	6
2 Appendix	8

Contribution Table

Table 1: Group Contribution and Marks

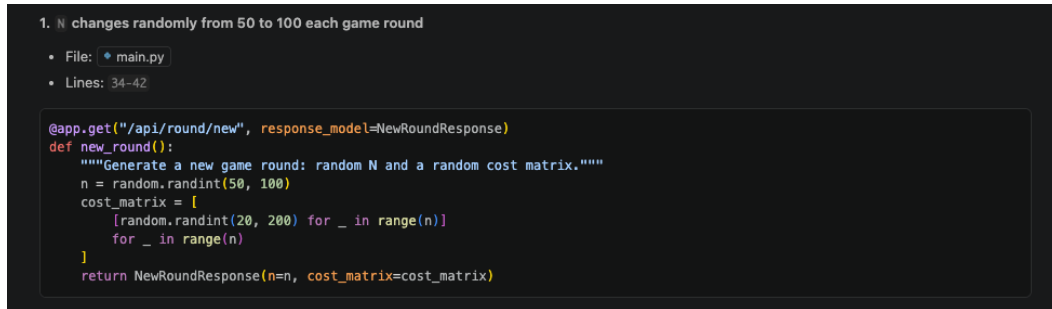
Index	NIBM Index	Marks /10	Contribution
16110762	COBSCCOMP242P-065	8	Backend – algorithms, database, API, unit tests
16116133	COBSCCOMP242P-075	7	Frontend – React UI, game screens, timing chart

1. Minimum Cost Problem

1.1. Functionality

1.1.1. Code Segment: N changes randomly from 50 to 100 in each game round

In our FastAPI backend, the number of workers and tasks N is picked randomly at the start of every round. We used Python's `random.randint` function to make sure N stays between 50 and 100.



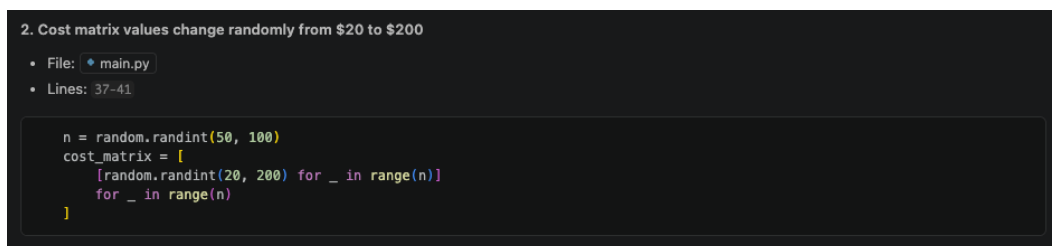
```
1. N changes randomly from 50 to 100 each game round
• File: main.py
• Lines: 34-42

@app.get("/api/round/new", response_model=NewRoundResponse)
def new_round():
    """Generate a new game round: random N and a random cost matrix."""
    n = random.randint(50, 100)
    cost_matrix = [
        [random.randint(20, 200) for _ in range(n)]
        for _ in range(n)
    ]
    return NewRoundResponse(n=n, cost_matrix=cost_matrix)
```

Figure 1: Code sample showing how N is randomly generated between 50 and 100.

1.1.2. Code Segment: Cost of assigning each task to each employee changing randomly from \$20 to \$200

Once N is decided, the app makes an N x N cost matrix. We used another `random.randint(20, 200)` call to fill each cell in the matrix. This makes every game round unique and different for the users.



```
2. Cost matrix values change randomly from $20 to $200
• File: main.py
• Lines: 37-41

n = random.randint(50, 100)
cost_matrix = [
    [random.randint(20, 200) for _ in range(n)]
    for _ in range(n)
]
```

Figure 2: This code part shows how cost values are set randomly from \$20 up to \$200.

1.1.3. Code Segment: Record in the database how long it takes to find the minimum cost using each algorithm

The system runs both of the algorithms and records the time it took. We used `time.perf_counter()` to get high precision timing in milliseconds. All this data gets saved into the `algorithm_results` table in SQLite database.

```

4. Record time taken for each algorithm in the database

• File: main.py
• Lines: 77-79, 109-117, 120-124

...
    t0 = time.perf_counter()
    sub_assignment, _, sub_steps = algo_fn(sub_matrix)
    elapsed_ms = (time.perf_counter() - t0) * 1000

    ...
    results.append(
        AlgorithmResult(
            algorithm_name=algo_name,
            assignment=full_assignment,
            total_cost=total_cost,
            time_ms=round(elapsed_ms, 4),
            steps=steps,
        )
    )
    ...
    round_id = save_round(
        n,
        [{"algorithm_name": r.algorithm_name, "total_cost": r.total_cost, "time_ms": r.time_ms} for r in results],
        player_name=body.player_name or None,
        user_total_cost=user_total_cost,
    )

```

Figure 3: Backend code that measures the speed of each algorithm.

1.1.4. Code Segment: Two different appropriate algorithms

The project uses two different ways to solve the assignment problem:

1. **Greedy Approach:** This is a fast $O(n^2)$ method. It doesn't always give the best answer but it is very quick.
2. **Hungarian Algorithm:** A more complex $O(n^3)$ method. This one is mathematically proven to always find the absolute minimum cost.

```

• Greedy algorithm implementation:
  • File: greedy.py
  • Lines: 14-67

def greedy_assignment(
    cost_matrix: list[list[int]],
) -> tuple[list[int], int, list[dict]]:
    ...
    for employee in range(n):
        best_task = -1
        best_cost = float("inf")
        candidates: list[tuple[int, int]] = []

        for task in range(n):
            if task not in assigned_tasks:
                candidates.append((task, cost_matrix[employee][task]))
                if cost_matrix[employee][task] < best_cost:
                    best_cost = cost_matrix[employee][task]
                    best_task = task

        assignment[employee] = best_task
        assigned_tasks.add(best_task)
        running_total += int(best_cost)
    ...

```

Figure 4: Our implementation of the Greedy logic.

```

• Hungarian algorithm implementation:
  ◦ File: hungarian.py
  ◦ Lines: 19-60

def hungarian_assignment(
    cost_matrix: list[list[int]],
) -> tuple[list[int], int, list[dict]]:
    """
    # Step 1 - Row reduction
    for i in range(n):
        row_min = min(matrix[i])
        for j in range(n):
            matrix[i][j] -= row_min

    # Step 2 - Column reduction
    for j in range(n):
        col_min = min(matrix[i][j] for i in range(n))
        for i in range(n):
            matrix[i][j] -= col_min

    # Iterate until minimum line cover equals n
    while True:
        row_cover, col_cover = _min_line_cover(matrix, n)
        if len(row_cover) + len(col_cover) >= n:
            break
        _adjust_matrix(matrix, n, row_cover, col_cover)

```

Figure 5: The implementation of the Hungarian algorithm.

1.1.5. Code Segment: Save player's name and their response in the database

When people play the game in Manual or Mixed modes, they can type in their name. The system then saves their name and the total cost they calculated. This is linked to the actual algorithm results to compare later.

```

5. Save player name to the database

• File: main.py
• Lines: 120-124, 140-149

round_id = save_round(
    n,
    [{"algorithm_name": r.algorithm_name, "total_cost": r.total_cost, "time_ms": r.time_ms} for r in results],
    player_name=body.player_name or None,
    user_total_cost=user_total_cost,
)

@app.post("/api/rounds/update-player-name")
def update_rounds_player_name(body: UpdatePlayerNameRequest):
    """Update player_name for a list of rounds (used when game ends)."""
    if not body.round_ids:
        raise HTTPException(status_code=400, detail="round_ids cannot be empty")
    if not body.player_name or not body.player_name.strip():
        raise HTTPException(status_code=400, detail="player_name cannot be empty")

    update_player_name(body.round_ids, body.player_name.strip())
    return {"status": "ok", "updated_rounds": len(body.round_ids)}

```

Figure 6: Code that handles saving the player name to the DB.

```
• Database save logic:
  ◦ File: database.py
  ◦ Lines: 43-58

def save_round(n: int, results: list[dict], player_name: str | None = None, user_total_cost: int | None = None) -> int:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO game_rounds (n, player_name, user_total_cost) VALUES (?, ?, ?)",
        (n, player_name, user_total_cost),
    )
    round_id = cursor.lastrowid
    cursor.executemany(
        "INSERT INTO algorithm_results (round_id, algorithm_name, total_cost, time_ms) VALUES (?, ?, ?, ?)",
        [(round_id, r["algorithm_name"], r["total_cost"], r["time_ms"]) for r in results],
    )
    conn.commit()
    conn.close()
    return round_id
```

Figure 7: The backend function used to persist the round results.

1.1.6. Database Table structure for the Game

We used two main tables to keep the game data organized:

- `game_rounds`: This stores general round info like the value of N, player name, and the time the round happened.
- `algorithm_results`: This table is linked to the rounds table and stores the cost and time for each specific algorithm.

```
• Database schema and normalized structure:
  ◦ File: database.py
  ◦ Lines: 16-31

CREATE TABLE IF NOT EXISTS game_rounds (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    n           INTEGER NOT NULL,
    player_name TEXT,
    user_total_cost INTEGER,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS algorithm_results (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    round_id    INTEGER NOT NULL REFERENCES game_rounds(id),
    algorithm_name TEXT NOT NULL,
    total_cost  INTEGER NOT NULL,
    time_ms     REAL NOT NULL
);
```

Figure 8: The SQLite database schema layout.

1.2. User Interface

1.2.1. UI screenshot for inputs and answers

The frontend was built using React and Vite. It has simple buttons for the users to choose how they want to play. You can either solve the problem manually by clicking cells in the matrix or let the computer do it for you.

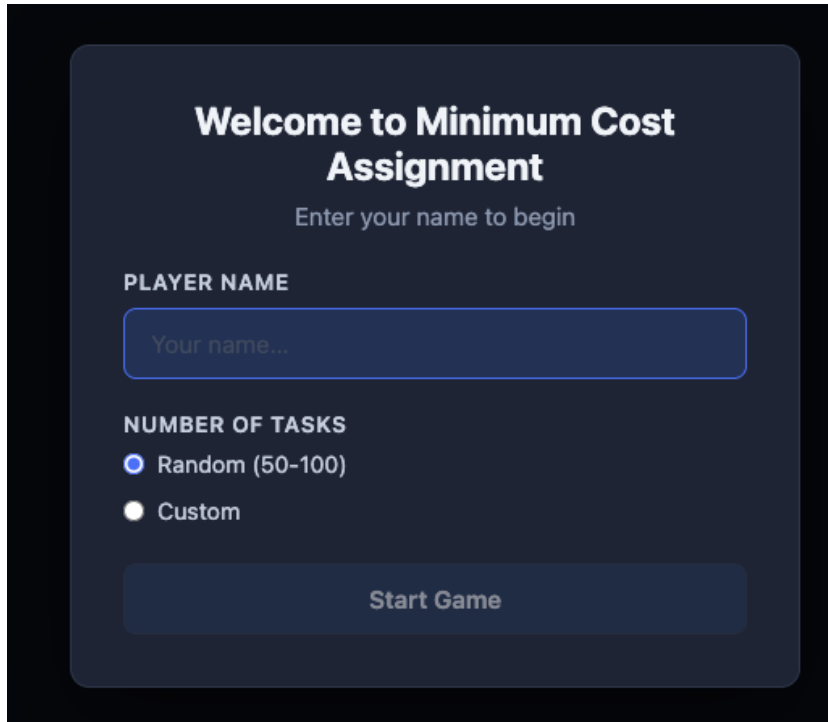


Figure 9: This shows where the user types their name before starting.

1.2.2. Validations and error handling

We added several checks to make the app stable:

- **Frontend (React):** We made the UI so it prevents common mistakes. For example, you can't assign one worker to two different tasks by accident because the UI will automatically toggle the previous choice.
- **Backend (FastAPI):** We use Pydantic models to check the data coming in. If the data is wrong or missing something, the backend sends an error instead of crashing. We also used try-except blocks in the database code to handle any SQL errors.

1.3. System Testing

1.3.1. Unit Testing

We used the pytest framework to test our code. We wrote several tests to make sure the algorithms work correctly. One important test was checking that the Hungarian algorithm always gives a cost that is less than or equal to the Greedy algorithm.

```
7 import pytest
8 from algorithms.greedy import greedy_assignment
9 from algorithms.hungarian import hungarian_assignment
10
11
12 # -----
13 # Helpers
14 # -----
15
16 def is_valid_assignment(assignment: list[int], n: int) -> bool:
17     """Every employee has a unique task in range [0, n)."""
18     return sorted(assignment) == list(range(n))
19
20
21 def is_valid_steps(steps: list[dict], n: int) -> bool:
22     """Steps list covers all n employees with required keys."""
23     if len(steps) != n:
24         return False
25     required = {"employee", "task"}
26     return all(required.issubset(s.keys()) for s in steps)
27
28
29 # -----
30 # Greedy tests
31 # -----
32
33 class TestGreedyAssignment:
34
35     def test_returns_valid_assignment_2x2(self):
36         matrix = [[9, 2], [3, 8]]
37         assignment, cost, steps = greedy_assignment(matrix)
38         assert is_valid_assignment(assignment, 2)
39         assert cost == sum(matrix[i][assignment[i]] for i in range(2))
40
41     def test_returns_valid_assignment_3x3(self):
42         matrix = [[4, 1, 3], [2, 0, 5], [3, 2, 2]]
43         assignment, cost, steps = greedy_assignment(matrix)
44         assert is_valid_assignment(assignment, 3)
45         assert cost == sum(matrix[i][assignment[i]] for i in range(3))
46
```

Figure 10: Testing our Greedy logic with pytest.

```

85 # -----
86 # Hungarian tests
87 # -----
88
89 class TestHungarianAssignment:
90
91     def test_optimal_2x2(self):
92         # Optimal: employee 0→task 1 (cost 2), employee 1→task 0 (cost 3) = 5
93         matrix = [[9, 2], [3, 8]]
94         assignment, cost, steps = hungarian_assignment(matrix)
95         assert is_valid_assignment(assignment, 2)
96         assert cost == 5
97
98     def test_optimal_3x3_known_solution(self):
99         # Optimal: 0→1 (1), 1→2 (1), 2→0 (1) = 3
100        matrix = [[3, 1, 2], [2, 3, 1], [1, 2, 3]]
101        assignment, cost, steps = hungarian_assignment(matrix)
102        assert is_valid_assignment(assignment, 3)
103        assert cost == 3
104
105    def test_optimal_4x4_known(self):
106        matrix = [
107            [9, 2, 7, 8],
108            [6, 4, 3, 7],
109            [5, 8, 1, 8],
110            [7, 6, 9, 4],
111        ]
112        # Brute-force verified optimal: 0→1 (2), 1→0 (6), 2→2 (1), 3→3 (4) = 13
113        assignment, cost, steps = hungarian_assignment(matrix)
114        assert is_valid_assignment(assignment, 4)
115        assert cost == 13
116
117    def test_returns_valid_assignment_single(self):
118        matrix = [[99]]
119        assignment, cost, steps = hungarian_assignment(matrix)
120        assert assignment == [0]
121        assert cost == 99
122
123    def test_all_same_cost(self):
124        matrix = [[7, 7, 7], [7, 7, 7], [7, 7, 7]]
125        assignment, cost, steps = hungarian_assignment(matrix)
126        assert is_valid_assignment(assignment, 3)
127        assert cost == 21

```

Figure 11: Testing the Hungarian algorithm for correctness.